

Лабораторная работа № 1

Обработка больших чисел - Вариант № 3

Техническое задание

Цель работы

Реализовать арифметическую операцию над числами, выходящими за разрядную сетку стандартных числовых типов, и выбрать структуры данных для их хранения и обработки.

Условие задачи

Смоделировать умножение целого числа длиной не более 40 десятичных цифр на действительное число в форме $\pm m.nE\pm K$. Суммарная длина мантиссы действительного числа не должна превышать 30 значащих цифр, а порядок должен содержать не более пяти цифр. Результат необходимо вывести в форме $\pm 0.m1E\pm K1$, где $m1$ содержит не более 40 значащих цифр, а $K1$ содержит не более пяти цифр.

Если точное произведение содержит больше 40 значащих цифр, необходимо округлить его по 41-й цифре. При некорректных исходных данных или выходе порядка результата за допустимые границы программа должна вывести сообщение об ошибке.

Исходные данные

Числа передаются через стандартный поток ввода, каждое на отдельной строке. Первым вводится целое число, вторым действительное.

Данные	Формат	Ограничения
Целое число	Необязательный знак + или -, затем десятичные цифры без пробелов. Можно использовать ведущие нули.	Не более 40 цифр; точка и порядок запрещены
Мантисса действительного числа	Необязательный знак, цифры и не более одной точки	Не более 30 значащих цифр по условию задания; должна присутствовать хотя бы одна цифра
Порядок действительного числа	E или e, необязательный знак +/- и цифры	От -99999 до +99999, не более пяти цифр в записи

Аварийные ситуации

Программа обрабатывает отсутствие строки во входном потоке, пустую строку, неверный формат числа, превышение длины целого числа, мантиссы или порядка, а также выход порядка результата за допустимый диапазон. Коды ошибок: `IO_ERR`, `EMPTY_INPUT_ERR`, `INVALID_FORMAT_ERR`, `BIG_INT_OVERFLOW_ERR`, `MANTISSA_OVERFLOW_ERR`, `EXPONENT_OVERFLOW_ERR`, `RESULT_EXPONENT_OVERFLOW_ERR`. Расшифровки ошибок указаны ниже.

Структуры данных

Представление целого числа

```
#define BIG_INT_MAX_DIGITS 40

typedef struct
{
    int sign;
    size_t len;
    unsigned char digits[BIG_INT_MAX_DIGITS];
} big_int_t;
```

Поле `sign` равно `1` для неотрицательного числа и `-1` для отрицательного. Поле `len` содержит число используемых элементов массива. Каждый элемент `digits` хранит числовое значение одной десятичной цифры от 0 до 9, а не её символьный код. Цифры расположены от старшей к младшей: число `123` представляется массивом `{1, 2, 3}` с длиной 3.

Ведущие нули удаляются после проверки длины входной записи. Ноль хранится как одна цифра `0` со знаком `1`.

Представление действительного числа

```
#define BIG_FLOAT_MAX_DIGITS 30
#define EXPONENT_MAX_DIGITS 5

typedef struct
{
    int sign;
    size_t len;
    int exponent;
```

```
    unsigned char digits[BIG_FLOAT_MAX_DIGITS];
} big_float_t;
```

Поля `sign`, `len` и `digits` описывают знак и цифры мантиссы. Поле `exponent` хранит порядок со знаком. Положение десятичной точки отдельно не хранится: она подразумевается перед первой цифрой массива.

Представление результата

```
#define RESULT_MAX_DIGITS 40

typedef struct
{
    int sign;
    size_t len;
    int exponent;
    unsigned char digits[RESULT_MAX_DIGITS];
} result_t;
```

Вспомогательные данные

Для точного промежуточного произведения используется массив `unsigned int product[70]`, поскольку произведение чисел из 40 и 30 цифр содержит не более 70 цифр. В функции разбора действительного числа используется массив `raw_digits[31]`.

Длины массивов и индексы имеют тип `size_t`. Коды ошибок описаны перечислением `error_t` в файле `inc/errors.h`.

Организация программы

Модуль	Назначение и основные функции
<code>src/main.c</code>	Последовательное выполнение ввода, вычисления и вывода в <code>main</code>
<code>src/big_int.c</code> , <code>inc/big_int.h</code>	Представление и разбор целого числа: <code>big_int_parse</code>
<code>src/big_float.c</code> , <code>inc/big_float.h</code>	Представление и нормализация действительного числа: <code>big_float_parse</code> , внутренняя функция <code>parse_exponent</code>
<code>src/calc.c</code> , <code>inc/calc.h</code>	Умножение: <code>multiply</code> ; внутренние функции <code>is_zero</code> , <code>multiply_digits</code> , <code>round_result</code>
<code>src/io.c</code> , <code>inc/io.h</code>	Приглашения, ввод и вывод: <code>print_intro</code> , <code>input_big_int</code> , <code>input_big_float</code> , <code>print_result</code> , <code>print_error</code>
<code>src/str.c</code> , <code>inc/str.h</code>	Чтение строки и проверка цифр: <code>read_line</code> , <code>is_digit</code>
<code>inc/errors.h</code>	Перечисление кодов завершения

Описание алгоритма

Чтение и проверка данных

Функция `read_line` читает строку целиком с помощью `getline`, удаляет конечный символ перевода строки `\n` и предшествующий ему `\r`, если он присутствует. Затем проверяет непустоту строки и возможность размещения в переданном буфере. Допустимая строка копируется вместе с завершающим нулевым символом, после чего динамическая память освобождается.

При разборе целого числа определяется знак, проверяется наличие только цифр и ограничение на их количество. После удаления ведущих нулей цифры переносятся в `big_int_t`. Для нулевого значения знак приводится к положительному.

При разборе действительного числа выполняются следующие действия:

1. Определяется знак мантиссы.
2. До символа `E`, `e` или конца строки читаются цифры и необязательная точка. Повторная точка и посторонние символы вызывают ошибку.
3. Подсчитываются общее количество цифр и количество цифр до точки.
4. При наличии экспоненциальной части разбираются знак порядка и не более пяти его цифр. После них должен следовать конец строки.
5. Находится первая ненулевая цифра мантиссы. Если все цифры нулевые, создаётся представление нуля с порядком 0.
6. Вычисляется нормализованный порядок, а ведущие и конечные нули исключаются из сохраняемой мантиссы.

Умножение мантисс

Если хотя бы один множитель равен нулю, результат сразу устанавливается равным `+0.0E+0`.

Для ненулевых множителей применяется умножение в столбик. Пусть целое число содержит `n` цифр, а мантисса действительного числа `m` цифр. Первые `n + m` элементов промежуточного массива обнуляются. Обход обоих массивов выполняется от младших цифр к старшим.

Для текущих индексов `i` и `j`, отсчитываемых в циклах от длины массива до 1, вычисляется:

```
pos = i + j - 1;
value = product[pos] + integer.digits[i - 1] * real.digits[j - 1];
product[pos] = value % 10;
product[pos - 1] += value / 10;
```

Остаток от деления на 10 сохраняется в текущем разряде, а частное переносится в старший. После завершения циклов ведущие нули произведения удаляются сдвигом цифр к началу массива.

Округление и вывод

Если точное произведение содержит не более 40 цифр, все они копируются в результат. В противном случае сохраняются первые 40 цифр и анализируется 41-я:

1. Если она меньше 5, оставшиеся цифры отбрасываются.
2. Если она не меньше 5, к 40-й цифре прибавляется единица.
3. Перенос распространяется в сторону старших разрядов через последовательность девяток.
4. Если все 40 сохранённых цифр были девятками, мантисса становится равной 0.1, а порядок увеличивается на единицу.

После округления удаляются конечные нули мантиссы и проверяется диапазон порядка. При допустимом порядке выводятся знак, 0., цифры мантиссы, E и порядок с явным знаком.

Оценка затрат

Основной алгоритм умножения выполняет $n * m$ шагов, поэтому его временная сложность составляет $O(n * m)$. Нормализация произведения и округление требуют $O(n + m)$.

Для промежуточного произведения требуется $O(n + m)$ дополнительной памяти. В текущей реализации размер массива фиксирован и равен 70 элементам `unsigned int`.

Тесты

Негативные тесты

Расшифровка кодов завершения:

Код	Имя	Значение
1	IO_ERR	Ошибка чтения, отсутствие строки или превышение размера буфера
2	EMPTY_INPUT_ERR	Введена пустая строка
3	INVALID_FORMAT_ERR	Неверный формат числа
4	BIG_INT_OVERFLOW_ERR	Целое число содержит более 40 цифр
5	MANTISSA_OVERFLOW_ERR	Мантисса содержит более 30 цифр
6	EXPONENT_OVERFLOW_ERR	Порядок содержит более пяти цифр
7	RESULT_EXPONENT_OVERFLOW_ERR	Порядок результата вне допустимого диапазона

№	Что проверяется	a	b	Полученный код
1	Пустой ввод целого числа	Пустая строка	Не читается	2
2	Пустой ввод действительного числа	1	Пустая строка	2
3	Отсутствие входных данных	EOF	Не читается	1
4	Отсутствие второй строки	1	EOF	1
5	Буква в целом числе	12a	1	3
6	Точка в целом числе	1.2	1	3
7	Порядок в целом числе	1E2	1	3
8	Знак без цифр	+	1	3
9	Две точки в мантиссе	1	1.2.3	3
10	Отсутствие цифр порядка	1	1E	3
11	Знак порядка без цифр	1	1E+	3
12	Запятая вместо точки	1	1,5	3
13	Пробел внутри числа	1	1 E2	3

Позитивные тесты

4 / 6

Позитивный тест № 18 проверяет одновременное достижение максимальной мантиссы результата (40 девяток) и максимального порядка **+99999**. Позитивный тест № 20 и негативный тест № 21 проверяют границу порядка после округления и нормализации: перенос через 40 девяток увеличивает порядок на единицу. В первом случае получается допустимый порядок **99999**, во втором — **100000**, поэтому программа должна вернуть код 7. Диапазон порядка результата проверяется после округления.

Для проведения операций над числами выходящими за рамки стандартного представления в компьютере, следует использовать специальные виды представления числа и алгоритмы длинной арифметики.

Контрольные вопросы

Битов	Без знака	Со знаком (дополнительный код)
8	0 ... 255	−128 ... 127
16	0 ... 65 535	−32 768 ... 32 767
32	0 ... 4 294 967 295	−2 147 483 648 ... 2 147 483 647
64	0 ... 18 446 744 073 709 551 615	−9 223 372 036 854 775 808 ... 9 223 372 036 854 775 807

Целые числа в пределах диапазона целого типа представляются точно. Для вещественных чисел точность определяется основанием системы счисления и количеством значащих разрядов мантиссы. Увеличение числа разрядов порядка расширяет диапазон, но само по себе не увеличивает число сохраняемых значащих цифр.

5 / 6

Формат	Битов под дробную часть	Точность мантиссы с учётом скрытого бита	Примерно значащих десятичных цифр
Двойная точность (double , 64 бита)	52	53 бита	16
Четверная точность (128 бит)	112	113 бит	34

3. Какие стандартные операции возможны над числами?

Над стандартными арифметическими типами определены сложение, вычитание, умножение, деление, унарные плюс и минус, сравнения. Для целых типов также определены целочисленное деление и деление с остатком.

4. Какой тип данных может выбрать программист, если обрабатываемые числа превышают возможный диапазон представления чисел в ПК?

Можно использовать массив цифр или блоков цифр, объединённый в структуру с длиной, знаком и, при необходимости, порядком. Массив может иметь фиксированный размер, если предельная длина известна, либо выделяться динамически.

5. Как можно осуществить операции над числами, выходящими за рамки машинного представления?

Операция разбивается на действия над отдельными цифрами или блоками, каждое из которых помещается в стандартный тип. Сложение и вычитание выполняются с переносами и заёмами, умножение с накоплением частичных произведений, деление с последовательным определением цифр частного. Знак и порядок обрабатываются отдельно.